

# **Desain Modular untuk Drone Pertanian Presisi: Integrasi Nanggro OS AI, Deteksi Hardware Otomatis, dan Aplikasi Pertanian Presisi**

Laporan riset ini menyajikan kerangka konseptual dan teknis yang komprehensif untuk pengembangan sebuah drone tiga baling otonom multifungsi. Proyek ini bertujuan untuk menciptakan sebuah platform robotika canggih yang tidak hanya mampu terbang tetapi juga beroperasi di darat dan di atas permukaan air, serta memiliki kemampuan untuk otomatisasi tugas-tugas pertanian presisi. Fondasi dari platform ini adalah Nanggro OS AI, sebuah sistem operasi robotika modular yang dirancang untuk mendukung deteksi perangkat keras otomatis, pembelajaran mandiri, dan operasi yang dapat diandalkan bahkan dalam kondisi tanpa koneksi internet. Arsitektur ini didukung oleh lapisan agen AI ganda, yaitu Hermes untuk perencanaan strategis tingkat tinggi dan PicoClaw untuk eksekusi taktis real-time, yang memastikan keselamatan dan efisiensi operasional. Seluruh sistem diintegrasikan dengan perangkat keras mikrokontroler Arduino yang fleksibel dan dilengkapi dengan sensor dan aktuator untuk navigasi, komunikasi, dan interaksi dengan lingkungan. Tujuan akhir dari penelitian ini adalah untuk merancang sebuah sistem yang cerdas, adaptif, dan mudah digunakan, yang dapat memberikan dampak nyata dalam bidang pertanian modern.

## **Arsitektur Sistem Operasi Nanggro OS AI: Deteksi Peralatan Otomatis dan Pembelajaran Mandiri**

Arsitektur inti dari proyek ini terletak pada Nanggro OS AI, sebuah sistem operasi robotika yang dirancang secara modular untuk mendukung fungsionalitas canggih seperti deteksi perangkat keras otomatis dan adaptasi sistem dinamis. Arsitektur ini merupakan fondasi yang memungkinkan drone untuk menjadi sebuah platform yang sangat fleksibel dan mudah dikonfigurasi ulang. Konsep "auto-detect hardware" bukanlah sekadar kemampuan untuk mengenali adanya perangkat, melainkan sebuah proses yang lebih kompleks di mana sistem secara proaktif mengidentifikasi jenis, fungsi, dan spesifikasi

perangkat yang terhubung, lalu menginisialisasi driver dan layanan yang sesuai secara otomatis <sup>1</sup>. Untuk mencapai hal ini, diperlukan lapisan abstraksi perangkat keras yang cerdas dan responsif. Salah satu mekanisme paling kuat untuk implementasi ini adalah melalui protokol Universal Serial Bus (USB). Chip SoC (System-on-a-Chip) seperti ESP32-S3, yang sering dipertimbangkan untuk proyek IoT dan robotika karena biaya rendah dan dukungan AI onboard, memiliki modul USB On-The-Go (OTG) yang sangat berguna <sup>19 27 37</sup>. Kemampuan OTG ini memungkinkan mikrokontroler untuk berfungsi sebagai host USB, sehingga dapat mengelola dan memindai perangkat-perangkat yang terhubung kepadanya, seperti kamera, sensor tambahan, atau penyimpanan eksternal <sup>32 49</sup>.

Pada saat boot-up atau saat perangkat baru dipasang, lapisan deteksi Nanggro OS AI akan melakukan pemindaian bus USB. Proses ini akan mengidentifikasi vendor (VID) dan identifikasi perangkat (PID) dari setiap perangkat yang terdeteksi. Informasi ini kemudian digunakan untuk mencocokkan perangkat dengan driver atau skrip inisialisasi yang telah ditentukan sebelumnya dalam sistem. Misalnya, jika sebuah kamera dengan VID/PID tertentu terhubung, sistem akan secara otomatis memuat driver untuk kamera tersebut dan mengaktifkan modul visi. Jika sebuah sensor GPS terdeteksi, maka modul navigasi akan diaktifkan; jika sebuah modul GSM terhubung, sistem akan siap untuk komunikasi jarak jauh <sup>1</sup>. Konsep desain modular ini sangat mirip dengan standar plug-and-play yang dikembangkan oleh Microsoft melalui proyek Jacdac, yang menyediakan kerangka kerja untuk komputasi fisik yang modular dengan protokol I2C, SPI, UART, dan USB <sup>22</sup>. Pendekatan Modular-Things juga menawarkan alat untuk membangun sistem fisik modular yang dapat dipasang dan dilepas dengan cepat <sup>26</sup>. Dengan mengadopsi prinsip-prinsip ini, Nanggro OS AI dapat meminimalkan konfigurasi manual yang rumit dan membuat sistem lebih mudah untuk dikembangkan dan dipelihara. Lapisan deteksi ini harus mampu mendukung juga protokol lain seperti serial, I2C, dan SPI untuk memastikan kompatibilitas dengan berbagai jenis sensor dan modul yang umum digunakan dalam proyek Arduino <sup>1</sup>.

Konsep "pembelajaran mandiri" dalam konteks Nanggro OS AI lebih tepat diartikan sebagai **adaptasi sistem dinamis**, bukan pembelajaran mesin dalam arti tradisional yang kompleks. Ini adalah kemampuan sistem untuk "belajar" dari pengalaman dan mengubah konfigurasinya untuk meningkatkan efisiensi atau kinerja. Contoh praktisnya adalah ketika sebuah perangkat baru terhubung, sistem tidak hanya menginisiasinya tetapi juga menyimpan preferensi atau kalibrasi yang diperlukan untuk perangkat tersebut. Dalam jangka panjang, sistem dapat belajar pola penggunaan dan mengoptimalkan alokasi sumber daya atau urutan inisialisasi. Namun, potensi yang lebih besar dari fitur ini terletak pada integrasi model AI on-device yang dapat beradaptasi secara langsung. Sebagai contoh, untuk fungsi penghindaran rintangan atau pelacakan wajah, model AI

yang berjalan di drone dapat "berlatih" dan meningkatkan akurasi setelah beberapa misi. Framework seperti TinyReptile menunjukkan bahwa *pembelajaran daring* pada perangkat kecil dapat dicapai dengan sedikit data, memungkinkan model untuk beradaptasi secara cepat tanpa perlu dikirim ke cloud untuk pelatihan ulang <sup>40</sup>. Ini membuka jalan bagi drone untuk secara bertahap meningkatkan kemampuan navigasinya di lingkungan yang berbeda-beda. Selain itu, kemampuan ini juga dapat diterapkan pada aspek lain, seperti optimasi penggunaan daya berdasarkan riwayat misi dan kondisi cuaca.

| Komponen             | Protokol Komunikasi                 | Contoh Implementasi dalam Nanggro OS AI                                                                                                                                                                      |
|----------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kamera               | USB, SPI, Serial                    | Saat terdeteksi via USB, sistem akan memuat driver V4L2 (Virtual Video 6 Layer 2) atau ekuivalennya, dan mulai menerima stream video untuk modul visi <sup>1</sup> <sup>28</sup> .                           |
| Sensor GPS           | Serial (UART)                       | Saat port serial terdeteksi, sistem akan mencari sinyal NMEA. Jika valid, modul navigasi akan diaktifkan dan koordinat GPS akan tersedia untuk semua agen <sup>1</sup> .                                     |
| Modul GSM            | Serial (AT Command)                 | Saat terdeteksi, sistem akan mengirimkan perintah AT dasar untuk memverifikasi konektivitas. Jika berhasil, modul komunikasi akan diaktifkan untuk sinkronisasi data atau notifikasi <sup>1</sup> .          |
| Panel Surya          | ADC (Analog-to-Digital Converter)   | Sensor tegangan/suhu pada panel surya akan dibaca melalui ADC. Data ini masuk ke modul manajemen daya untuk menentukan apakah perlu mengaktifkan mode hemat energi atau mengisi ulang baterai <sup>2</sup> . |
| Roda (Operasi Darat) | GPIO (General-Purpose Input/Output) | Saat roda terhubung, pin GPIO yang sesuai akan dikonfigurasi sebagai input/output. Modul land-based controller akan diaktifkan, dan PID controller untuk stabilitas akan disiapkan <sup>1</sup> .            |

Untuk mengelola seluruh sistem yang kompleks ini, Nanggro OS AI mengusulkan arsitektur multi-agennya yang terdiri dari dua agen spesialis: Hermes dan PicoClaw <sup>1</sup>. Arsitektur hierarkis ini, yang mengadopsi model otak-kerebelum, memisahkan tugas-tugas yang berbeda secara logis untuk meningkatkan keamanan, keandalan, dan efisiensi. Hermes berfungsi sebagai "otak" strategis, bertanggung jawab atas perencanaan tingkat tinggi. Ia menerima instruksi dari pengguna (misalnya, "Petakan sawah di barat dan tabur pupuk di timur"), menggunakan kemampuan pemahaman bahasa alami (mungkin dengan bantuan LLM lokal), dan menerjemahkannya menjadi rencana misi yang terstruktur. Rencana ini mencakup langkah-langkah seperti menghitung jalur penerbangan optimal, menentukan titik-titik penjemputan, dan mengatur alokasi sumber daya <sup>1</sup>. Arsitektur ini sejalan dengan pendekatan di mana LLM digunakan untuk merancang mesin status hingga yang kompleks untuk robotik, memastikan kepatuhan terhadap protokol operasional <sup>36</sup>.

Di sisi lain, PicoClaw bertindak sebagai "kerebelum" taktis, yang merupakan lapisan kontrol deterministik dan waktu nyata. Tugas utamanya adalah menerima perintah-perintah yang lebih sederhana dari Hermes dan mengubahnya menjadi sinyal motorik spesifik untuk mengontrol motor baling-baling, servo, dan aktuator lainnya <sup>1</sup>. Penting

untuk dicatat bahwa lapisan PicoClaw harus bersifat deterministik untuk memastikan respons yang cepat dan stabil dalam menjaga keseimbangan drone. Memisahkan proses non-deterministik yang mungkin dilakukan oleh LLM atau model AI lainnya dari lapisan kontrol rendah ini adalah prinsip desain yang krusial untuk keselamatan operasional <sup>1</sup>. Komunikasi antara kedua agen ini adalah titik kritis yang menentukan efektivitas seluruh sistem. Protokol publikasi-subskripsi (publish-subscribe) yang populer di dunia robotika, seperti yang digunakan oleh Robot Operating System (ROS), adalah solusi yang mapan dan terbukti untuk tujuan ini <sup>6</sup>. Meskipun ROS tradisional berjalan di atas sistem operasi Linux, versi mikro yang lebih ringan seperti Micro-ROS dirancang khusus untuk lingkungan berdaya terbatas seperti mikrokontroler <sup>6</sup>. Micro-ROS dapat menjadi middleware ideal untuk menghubungkan Hermes dan PicoClaw, bahkan jika mereka berjalan pada mikrokontroler yang sama atau berbeda, dengan menggunakan middleware Micro XRCE-DDS untuk komunikasi efisien melalui UDP <sup>6</sup>. Framework lain seperti RoboNeuron juga menunjukkan arah masa depan dalam menghubungkan agen yang didorong LLM dengan middleware robotika seperti ROS2, yang dapat memberikan inspirasi arsitektural untuk implementasi Nanggro OS AI <sup>33 46</sup>. Dengan demikian, arsitektur Nanggro OS AI tidak hanya menyediakan kemudahan konfigurasi melalui deteksi hardware, tetapi juga memberikan fondasi logis yang kokoh untuk membangun sistem robotika otonom yang canggih, aman, dan adaptif.

## Integrasi Perangkat Keras dan Multi-Moda Operasional

Integrasi perangkat keras yang solid adalah tulang punggung dari drone multifungsi ini, memungkinkannya untuk beroperasi secara efektif di tiga domain: udara, darat, dan air. Fondasi kontrolnya adalah mikrokontroler Arduino, yang dikenal karena popularitasnya, komunitas yang luas, dan biaya rendah. Namun, untuk mendukung kebutuhan komputasi yang lebih tinggi dari Nanggro OS AI, terutama untuk fungsi-fungsi AI on-device, penggunaan mikrokontroler yang lebih kuat seperti chip ESP32-S3 direkomendasikan. ESP32-S3 menawarkan kinerja CPU yang lebih tinggi, RAM yang lebih besar, dan yang terpenting, dukungan bawaan untuk USB OTG (On-The-Go) dan konektivitas nirkabel Wi-Fi/Bluetooth, yang sangat penting untuk komunikasi dan deteksi hardware <sup>19 27 37</sup>. Mikrokontroler ini dapat bertindak sebagai unit pemrosesan utama yang mengelola komunikasi dengan Arduino (yang mungkin lebih difokuskan pada kontrol motorik yang sangat cepat) dan semua sensor lainnya. Platform ini dirancang dengan pendekatan modular, di mana setiap komponen—mulai dari motor dan baling-baling, kamera, GPS, modul GSM, hingga mekanisme pelepas beban—dapat dihubungkan dan dikonfigurasi secara fleksibel.

Navigasi dan posisi absolut ditopang oleh modul GPS yang andal. Untuk aplikasi pertanian presisi, akurasi sangat krusial. Oleh karena itu, penggunaan receiver GPS yang mendukung teknologi Real-Time Kinematic (RTK) sangat disarankan. Teknologi RTK memungkinkan pencapaian akurasi posisi centimeter-level dengan memanfaatkan sinyal koreksi dari stasiun referensi lokal atau satelit, yang secara drastis mengurangi kebutuhan akan Ground Control Points (GCPs) dan meningkatkan keandalan peta yang dihasilkan <sup>12</sup>. Modul GPS akan terhubung ke mikrokontroler utama melalui antarmuka serial (UART), mentransmisikan data koordinat NMEA yang dapat diproses oleh modul navigasi di dalam Nanggro OS AI <sup>1</sup>. Untuk penghindaran rintangan dan pelacakan objek, kamera RGB atau kamera monokrom dengan resolusi yang cukup tinggi akan digunakan. Kamera ini akan terhubung melalui USB, memungkinkan transfer data gambar yang cepat ke mikrokontroler utama untuk pemrosesan citra waktu nyata, seperti deteksi wajah atau penghalang <sup>1</sup>. Konektivitas jarak jauh dan kontrol dari jarak jauh dijamin oleh modul GSM/GPRS. Modul ini memungkinkan drone untuk tetap terhubung ke jaringan internet seluler, yang esensial untuk operasi remote melalui Telegram, sinkronisasi data, dan pembaruan firmware <sup>1</sup>. Modul ini akan terhubung melalui UART dan akan dikelola oleh agen Hermes untuk mengkoordinasikan komunikasi.

Fitur unik dari proyek ini adalah kemampuannya untuk beroperasi di darat dan di air. Untuk operasi darat, set sistem ban baja yang ringan akan dipasang pada rangka drone. Ban ini akan dilengkapi dengan motor independen yang dikendalikan oleh servo atau motor DC, yang diatur oleh mikrokontroler. Ketika drone mendarat, PicoClaw akan beralih ke modus kontrol land-based, menggunakan data dari sensor seperti IMU (Inertial Measurement Unit) untuk menjaga keseimbangan dan menggerakkan drone maju, mundur, atau berbelok. Untuk operasi di air, sistem pelampung (floatation system) yang ringan namun kuat akan dipasang di bawah fuselage. Pelampung ini harus dirancang agar tidak mengganggu aerodinamika saat terbang. Ketika drone mendarat di permukaan air, sistem akan beralih ke modus aquatic, di mana PicoClaw akan mengontrol servos atau motor kecil untuk menjaga stabilitas dan orientasi drone di atas air. Fleksibilitas ini memungkinkan drone untuk melakukan survei di area yang sulit diakses seperti rawa-rawa atau pinggir sungai, yang tidak dapat dicapai oleh drone udara biasa. Setiap perubahan moda operasional akan dipicu secara otomatis oleh sensor yang mendeteksi kontak dengan permukaan atau perubahan gaya angkat.

Manajemen daya adalah aspek kritis lainnya untuk memastikan operasi yang berkelanjutan, terutama untuk misi di daerah terpencil. Sistem ini akan menggunakan baterai LiPo berkapasitas tinggi sebagai sumber daya utama. Untuk meningkatkan ketahanan, panel surya kecil akan dipasang pada permukaan drone. Panel ini akan terhubung ke sistem pengisian daya pintar yang dapat mengisi ulang baterai LiPo saat

drone berada di bawah sinar matahari <sup>2</sup>. Sistem ini tidak dimaksudkan untuk menggerakkan drone secara terus-menerus dengan tenaga surya, tetapi sebagai cadangan daya darurat. Dalam kondisi darurat, seperti baterai utama drop secara drastis, panel surya dapat menyediakan daya minimal untuk menjaga hidup sistem komando dan kontrol (GPS, komunikasi GSM) agar drone dapat kembali ke titik asal (Return-to-Home) <sup>1</sup>. Integrasi sistem pengisian baterai solar yang cerdas, termasuk penggunaan teknik Maximum Power Point Tracking (MPPT), akan sangat meningkatkan efisiensi dan keandalan sistem daya secara keseluruhan <sup>2</sup>. Selain itu, fitur Return-to-Home (RTH) yang andal adalah fitur keselamatan mutlak. Fitur ini akan dipicu oleh beberapa kondisi: hilangnya sinyal dari remote control, baterai yang levelnya turun di bawah ambang batas yang ditentukan, atau kesalahan kritis pada sistem navigasi <sup>1</sup>. Logika RTH akan dieksekusi oleh PicoClaw untuk memastikan respons yang cepat dan deterministik, mengarahkan drone kembali ke koordinat awal menggunakan data dari modul GPS dan menjaga keseimbangan selama proses penerbangan kembali. Semua komponen ini—dari mikrokontroler yang kuat hingga sistem pelampung dan panel surya—harus diintegrasikan secara erat dengan Nanggro OS AI, yang bertindak sebagai pusat kendali yang mengkoordinasikan semuanya untuk menciptakan platform drone yang benar-benar multifungsi dan tangguh.

## Implementasi Kecerdasan Buatan Lokal (LLM) dan Komputasi Ujung

Integrasi Kecerdasan Cerdas Makhluk (LLM) lokal merupakan salah satu inovasi paling signifikan dalam arsitektur Nanggro OS AI, memungkinkan drone untuk memahami dan merespons perintah dalam bahasa alami, serta memiliki potensi untuk adaptasi cerdas. Namun, implementasi LLM di perangkat dengan sumber daya terbatas seperti mikrokontroler adalah tantangan teknis yang besar. Keberhasilan proyek ini bergantung pada pemilihan model yang tepat, format penyimpanan yang efisien, dan runtime inferensi yang dioptimalkan. Salah satu pilihan model yang paling relevan dan menjanjikan adalah Phi-3-mini, sebuah model dengan parameter 3,8 miliar yang telah dirancang secara khusus untuk tugas-tugas mengikuti instruksi, pemahaman umum, dan penalaran matematis, namun tetap ringan <sup>16</sup>. Model ini tersedia dalam format GGUF, sebuah format model terkuantisasi yang dioptimalkan untuk inferensi di perangkat dengan memori terbatas, seperti telepon seluler atau mikrokontroler <sup>17 24</sup>. Format GGUF ini sangat cocok untuk persyaratan "flash to disk" dari Nanggro OS AI, karena

memungkinkan model AI yang kuat untuk diinstal dan diperbarui melalui media penyimpanan eksternal seperti flashdisk atau kartu SD <sup>1</sup> .

Proses menjalankan LLM di perangkat seperti ESP32-S3 memerlukan serangkaian optimasi yang cermat. Pertama, model Phi-3-mini akan melalui proses *quantization*, yaitu proses penurunan presisi numerik (misalnya, dari 16-bit floating-point ke 8-bit integer). Meskipun ini menyebabkan sedikit penurunan akurasi, penurunan ini sering kali dapat diterima demi keuntungan besar dalam ukuran model dan kecepatan inferensi. Kedua, diperlukan *runtime inference* yang efisien. Framework seperti vLLM, yang dirancang untuk efisiensi inferensi, dapat menjadi acuan untuk mengembangkan runtime khusus yang dioptimalkan untuk perangkat embedded <sup>16</sup> . Optimasi penting lainnya adalah manajemen cache KV (Key-Value). Pada LLM, setiap token yang dihasilkan bergantung pada representasi dari semua token sebelumnya, yang disimpan dalam cache KV. Mengelola cache ini secara cerdas, misalnya dengan menggunakan chunk-wise, global optimization, dapat secara signifikan mengurangi latensi dan penggunaan memori <sup>14</sup> . Pengalaman dari implementasi LLM on-device di Android menunjukkan bahwa tantangan utama adalah performa dan penggunaan memori, yang memerlukan pendekatan yang sangat hati-hati dalam desain sistem <sup>24 41</sup> .

Peran LLM dalam Nanggro OS AI bukanlah untuk mengontrol drone secara langsung, karena proses inferensi LLM tidak bersifat deterministik dan tidak dapat dijamin waktu eksekusinya. Sebaliknya, LLM berfungsi sebagai "interpreter" atau "planner" tingkat tinggi yang bekerja sama dengan agen Hermes. Arsitektur kerjanya akan seperti berikut: 1.

**Input Perintah:** Pengguna memberikan perintah dalam bahasa alami melalui antarmuka Telegram (teks) atau dengan merekam suara (suara catatan) <sup>1</sup> . 2. **Preprocessing Suara (Opsional):** Jika perintah datang dari voice note, audio tersebut harus diproses.

Rekognisi suara pada Arduino sendiri sangat sulit. Solusi yang lebih praktis adalah mengekstrak fitur-fitur penting dari sampel suara (seperti energi total per band frekuensi dan Zero Crossing Rate) dan mengirimkannya ke mikrokontroler yang lebih kuat untuk diproses <sup>4</sup> . Alternatif lain adalah mengirimkan seluruh rekaman audio ke server

backend (Python script) yang kemudian mengonversi teks ke suara dan mengirimkannya ke LLM. 3. **Pemahaman Instruksi:** Perintah teks yang sudah ada akan diberikan kepada LLM lokal yang berjalan di Nanggro OS AI. Dengan prompt engineering yang baik, LLM akan mampu mengekstrak entitas kunci dari perintah, seperti aksi yang diminta

("petakan", "tabur"), lokasi target ("sawah barat", "area timur"), dan parameter lainnya ("seluas 2 hektar"). 4. **Generasi Rencana:** Berdasarkan pemahaman ini, LLM akan menghasilkan deskripsi rencana misi yang terstruktur dalam format yang dapat dipahami

oleh Hermes, misalnya dalam format JSON atau pseudocode robotika. 5. **Eksekusi oleh**

**Hermes:** Agen Hermes akan mem-parsing output dari LLM, mengubahnya menjadi



serangkaian tugas-tugas yang konkret (misalnya, "jalankan fungsi petak\_sawah(luas=20000, lokasi='barat')", "jalankan fungsi\_tabur\_pupuk(koordinat=[...])"), dan kemudian menugaskannya kepada PicoClaw atau modul lain yang relevan. Contohnya, jika pengguna mengirim pesan ke bot Telegram: "Buat peta sawah di barat, lalu tabur pupuk di area timur," Hermes menerima pesan tersebut. Hermes kemudian menggunakan LLM lokal untuk memahami maksud, mengidentifikasi entitas seperti 'area timur' dan 'barat', serta tindakan 'buat peta' dan 'tabur pupuk'. Berdasarkan pemahaman itu, Hermes membuat rencana strategis: 1) Jalur petak sawah, 2) Titik-titik drop pupuk <sup>1</sup>. Rencana ini kemudian dikonversi menjadi serangkaian perintah taktis yang lebih sederhana dan ditransmisikan ke PicoClaw. Pendekatan ini memanfaatkan kekuatan LLM untuk pemahaman bahasa tanpa mengorbankan keandalan kontrol rendah yang diberikan oleh lapisan PicoClaw. Potensi untuk "pembelajaran mandiri" juga dapat dieksplorasi di sini. Misalnya, sistem dapat mencatat hasil dari berbagai rencana yang dihasilkan LLM dan mengevaluasi keberhasilannya. Data ini kemudian dapat digunakan untuk melatih ulang atau mengkalibrasi LLM lokal dalam siklus offline, memungkinkan sistem untuk "belajar" dari pengalamannya dan meningkatkan kualitas perencanaannya dari waktu ke waktu.

| Komponen AI        | Perangkat Keras Target    | Model/Format                                         | Peran Utama dalam Nanggroe OS AI                                                                                                            | Tantangan Utama                                                                              |
|--------------------|---------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| LLM Bahasa         | ESP32-S3 / Host Processor | Phi-3-mini (GGUF) <sup>16</sup>                      | Pemrosesan instruksi natural language, interpretasi perintah dari pengguna (Telegram/Suara), generasi rencana misi kasar untuk agen Hermes. | Performa inferensi lambat, penggunaan memori tinggi, latency tinggi.                         |
| Face Tracking      | ESP32-S3 + Camera         | YOLOv5-based model (contoh: RICE-YOLO) <sup>10</sup> | Deteksi dan pelacakan wajah manusia untuk interaksi atau pengawasan.                                                                        | Latency inferensi harus < 33ms untuk frame rate 30fps, ukuran model harus ringan.            |
| Obstacle Avoidance | ESP32-S3 + Depth Sensor   | Custom CNN or TFLM-based model <sup>18</sup>         | Deteksi rintangan di dekat drone untuk navigasi aman.                                                                                       | Butuh model yang sangat cepat dan akurat, bekerja dengan data dari sensor ultrasonik/kamera. |
| Online Learning    | ESP32-S3                  | TinyReptile-inspired algorithm <sup>40</sup>         | Adaptasi model AI (misalnya, penghindaran rintangan) dengan data baru dari misi sebelumnya.                                                 | Membutuhkan memori non-volatile untuk menyimpan model, algoritma adaptasi yang efisien.      |

Selain LLM bahasa, AI lokal juga akan digunakan untuk tugas-tugas waktu nyata seperti penghindaran rintangan dan pelacakan wajah. Untuk ini, model-model yang lebih kecil dan dioptimalkan seperti versi yang dimodifikasi dari arsitektur YOLO (You Only Look Once) akan digunakan. Model seperti RICE-YOLO, yang dirancang untuk mendeteksi spike padi, menunjukkan bagaimana arsitektur YOLO dapat dimodifikasi untuk meningkatkan akurasi pada target kecil dan padat, meskipun dengan sedikit penurunan kecepatan <sup>10</sup>. Untuk aplikasi seperti ini, model harus di-optimize agar dapat berjalan pada kecepatan inferensi yang tinggi (misalnya, >30 FPS) untuk memastikan respons



yang lancar. TensorFlow Lite Micro (TFLM) adalah kerangka kerja yang sangat baik untuk mengimplementasikan model-model ini di perangkat embedded seperti ESP32-S3 <sup>18</sup>. Dengan menggabungkan berbagai model AI lokal yang dioptimalkan ini, Nanggro OS AI dapat memberikan kemampuan visi dan navigasi yang cerdas secara langsung di drone, mengurangi ketergantungan pada koneksi internet dan mempercepat waktu respons.

## Fungsionalitas Pertanian Presisi: Pemetaan dan Penyebaran Material

Aplikasi pertanian presisi adalah penerapan fungsional yang paling signifikan dari drone multifungsi ini, mengubahnya dari sekadar platform teknologi menjadi alat produktif yang dapat memberikan nilai ekonomi dan lingkungan. Fungsionalitas ini mencakup dua tugas utama: pemetaan lahan secara otomatis untuk mendapatkan informasi tentang kondisi sawah, dan penyebaran material seperti pupuk secara presisi. Kedua tugas ini memerlukan integrasi yang cermat antara perangkat keras, perangkat lunak navigasi, dan algoritma analisis data.

Untuk pemetaan sawah dan pengukuran lahan, drone akan dilengkapi dengan kamera optikal berkualitas tinggi. Proses pemetaan akan didasarkan pada teknik fotogrametri Struktur dari Gerakan (SfM), di mana serangkaian foto yang tumpang tindih tinggi diambil dari ketinggian yang konstan <sup>11</sup>. Foto-foto ini kemudian diolah oleh perangkat lunak (baik di drone atau di backend) untuk membangun model 3D permukaan lahan, yang selanjutnya dapat digunakan untuk menghasilkan peta ortofoto (gambar udara yang telah dikoreksi distorsi) dan Digital Elevation Models (DEM) <sup>11</sup>. Akurasi dari pemetaan ini sangat bergantung pada beberapa faktor kunci. Pertama, Ground Sample Distance (GSD), yang merupakan jarak di tanah yang diwakili oleh satu piksel dalam citra. GSD yang lebih kecil (resolusi lebih tinggi) memungkinkan deteksi detail yang lebih baik <sup>12</sup>. GSD dapat dihitung berdasarkan ketinggian penerbangan dan spesifikasi sensor kamera. Kedua, akurasi spasial dari citra sangat bergantung pada kualitas sistem navigasi. Penggunaan receiver GPS yang mendukung koreksi RTK (Real-Time Kinematic) adalah standar industri untuk mencapai akurasi centimeter-level, yang secara signifikan mengurangi atau menghilangkan kebutuhan akan Ground Control Points (GCPs) yang melelahkan untuk ditempatkan di lapangan <sup>12</sup>. Dengan RTK, koordinat setiap foto yang diambil dapat ditentukan dengan sangat akurat, memastikan bahwa peta yang dihasilkan berada pada skala dan posisi yang benar di dunia nyata.

Setelah peta terbentuk, analisis lebih lanjut dapat dilakukan. Salah satu aplikasi yang paling berguna adalah deteksi dan pemetaan lahan sawah. Algoritma seperti PPPM (Phenology- and Pixel-Based Paddy Rice Mapping) dapat direplikasi di Nanggro OS AI. Algoritma ini bekerja dengan mendeteksi sinyal genangan air di lahan sawah selama fase transplantasi, yang merupakan karakteristik unik padi yang tidak dimiliki oleh tanaman lainnya <sup>9</sup>. Dengan menganalisis citra multispektral (jika kamera yang digunakan mendukungnya) dan menggunakan indeks seperti LSWI (Land Surface Water Index), sistem dapat secara otomatis mengidentifikasi area yang ditanami padi dan menghitung luasannya. Proses ini juga mempertimbangkan fenologi tanaman, menggunakan data suhu permukaan bumi (LST) untuk menentukan periode transplantasi yang paling mungkin <sup>9</sup>. Hasil akhirnya adalah peta digital yang menunjukkan distribusi lahan sawah, yang dapat digunakan oleh petani untuk perencanaan, subsidi, atau pemantauan. Selain pemetaan lahan, sistem juga dapat menghitung indeks vegetasi seperti NDVI (Normalized Difference Vegetation Index) dari citra optikal untuk menilai kesehatan tanaman dan mendeteksi area yang kurang subur.

Aspek kedua dari aplikasi pertanian presisi adalah penyebaran material, seperti pupuk atau benih. Sistem ini akan dilengkapi dengan mekanisme pelepas beban yang andal. Berdasarkan desain sistem item dropping yang telah terbukti, mekanisme ini dapat dibangun menggunakan mikrokontroler seperti Arduino Nano dan servo digital (misalnya, TowerPro MG90D) yang dikendalikan nirkabel <sup>13</sup>. Servo ini akan menggerakkan latch yang menahan beban. Ketika perintah pelepasan diterima, servo akan berputar untuk membuka latch, memungkinkan material untuk jatuh keluar. Komunikasi untuk mengaktifkan mekanisme ini dapat terjadi melalui koneksi serial nirkabel (misalnya, HC-12 433 MHz) dari remote control, atau lebih canggih lagi, secara otomatis oleh agen Hermes berdasarkan rencana misi yang telah ditentukan <sup>13</sup>. Untuk penyebaran presisi, drone harus mampu membuka mekanisme pelepas beban pada koordinat GPS tertentu yang telah ditentukan dalam rencana misi. Ini memerlukan koordinasi yang sempurna antara modul navigasi (yang mengetahui posisi drone saat ini) dan modul eksekusi payload. Ketika drone mencapai titik target, PicoClaw akan mengirimkan sinyal ke servo untuk membuka pelepas beban.

Namun, implementasi penyebaran material ini menghadapi beberapa tantangan teknis. Variabilitas material (misalnya, butiran pupuk yang bisa macet) dan kondisi aerodinamika saat drone terbang dapat mempengaruhi akurasi penempatan. Studi kasus penggunaan drone untuk mengisi ulang sensor tag menyoroti tantangan lain, yaitu interferensi elektromagnetik (EMI) dari motor drone yang dapat mengganggu komunikasi nirkabel internal, seperti antara drone dan gateway telepon seluler <sup>8</sup>. Meskipun studi tersebut berfokus pada komunikasi, EMI yang sama dapat memengaruhi

keandalan komponen elektronik sensitif lainnya, termasuk modul radio untuk pengendalian dan sistem pelepas beban. Oleh karena itu, desain PCB yang baik dengan isolasi dan filtering yang memadai akan menjadi krusial untuk memastikan operasi yang andal. Meskipun demikian, dengan perancangan yang cermat, kemampuan untuk melakukan pemetaan dan penyebaran material secara presisi akan memberikan nilai tambah yang sangat besar, memungkinkan petani untuk menerapkan prinsip-prinsip pertanian presisi yang sebelumnya hanya dapat diakses oleh alat berat atau pesawat terbang kargo miniatur.

## Antarmuka Pengguna dan Sistem Kontrol Interoperabel

Kemudahan penggunaan dan fleksibilitas kontrol adalah pilar fundamental dari Nangroe OS AI, dirancang untuk memastikan sistem ini dapat diakses dan dikendalikan oleh berbagai jenis pengguna, baik di lingkungan terhubung maupun terisolasi. Arsitektur antarmuka ini berpusat pada interoperabilitas, menawarkan beberapa jalur komunikasi yang saling melengkapi, yang semuanya diatur dan diproses oleh agen ganda Hermes dan PicoClaw. Antarmuka utama yang diusulkan adalah melalui Bot Telegram, dashboard web, perintah suara (catatan suara dan sintesis ucapan), serta aplikasi Android, menciptakan ekosistem kontrol yang lengkap dan intuitif.

Bot Telegram adalah salah satu antarmuka yang paling kuat dan populer untuk kontrol jarak jauh. Arsitektur ini memungkinkan pengguna untuk mengirimkan perintah dan menerima status drone dari mana pun dengan konektivitas data seluler. Prosesnya melibatkan beberapa komponen: pertama, bot Telegram dibuat menggunakan BotFather di aplikasi Telegram, yang menghasilkan API token unik [5](#). Kemudian, sebuah server backend (misalnya, skrip Python di Raspberry Pi atau server cloud) menggunakan library seperti `telepot` untuk berkomunikasi dengan API Telegram, mendengarkan pesan masuk dari pengguna [5](#). Server ini bertindak sebagai perantara. Ketika pengguna mengirim pesan (misalnya, `/takeoff`, `petakan sawah`), server menerjemahkan pesan tersebut dan mengirimkannya ke drone. Komunikasi antara server dan drone dapat dilakukan melalui berbagai medium, seperti koneksi serial langsung (jika server berdekatan) atau melalui modul GSM/GPRS yang terintegrasi di drone, yang memungkinkan komando dari jarak jauh [1](#). Agen Hermes di drone akan menerima komando ini dan memprosesnya sesuai dengan logika yang telah ditentukan.

Sebagai alternatif atau pelengkap, sebuah dashboard web dapat dibangun untuk memberikan visualisasi yang kaya dan kontrol yang lebih terperinci. Dashboard ini dapat

menampilkan peta lokasi drone secara waktu nyata, status baterai, kualitas sinyal GPS, serta hasil pemetaan yang sedang berlangsung. Pengguna dapat merencanakan misi dengan mengklik di atas peta untuk menentukan titik-titik tujuan atau area survei. Dashboard ini juga dapat menampilkan data historis misi dan statistik kinerja drone. Dengan menggunakan database lokal SQLite di drone, semua data misi dan metadata dapat disimpan dan diakses melalui dashboard saat terhubung, sesuai dengan prinsip operasi offline-first <sup>1</sup>.

Fitur perintah suara menambahkan lapisan interaktivitas yang sangat intuitif. Pengguna dapat merekam perintah verbal (notepad suara) dan mengirimkannya melalui Telegram atau aplikasi Android. Di sisi drone, sistem akan menerima rekaman suara ini. Seperti yang dibahas sebelumnya, pemrosesan suara yang kompleks pada Arduino sangat tidak efisien. Oleh karena itu, rekaman suara ini akan dikirim ke unit pemrosesan yang lebih kuat (misalnya, ESP32-S3 atau host processor) untuk diubah menjadi teks menggunakan layanan Speech-to-Text (STT). Teks ini kemudian diberikan kepada LLM lokal untuk pemahaman instruksi, dan hasilnya dikembalikan kepada pengguna. Untuk umpan balik, sistem akan menggunakan Text-to-Speech (TTS). Setelah menerima perintah, Nangroe OS AI dapat menggunakan TTS untuk memberikan konfirmasi verbal, misalnya "Siap menerima perintah selanjutnya." Umpan balik verbal ini sangat berguna di lingkungan terpencil di mana pengguna mungkin tidak dapat memeriksa layar secara visual. Implementasi STT/TTS dapat diintegrasikan dengan baik, di mana output dari LLM (yang berupa teks) langsung diinputkan ke dalam engine TTS.

Terakhir, aplikasi Android yang khusus dibuat akan menyediakan antarmuka sentuh yang paling intuitif. Aplikasi ini dapat mengintegrasikan semua fungsi yang ada di dashboard web, ditambah dengan kontrol sentuh yang mudah digunakan. Pengguna dapat mengontrol gerakan dasar drone (naik/turun, maju/mundur, berputar) dengan joystick virtual, serta memicu misi preseden (seperti takeoff, landing, RTH) dengan satu ketukan tombol. Aplikasi ini juga dapat menggunakan sensor smartphone (GPS, kamera) untuk tugas-tugas tambahan, seperti menentukan lokasi pengambilan gambar atau merekam video. Semua antarmuka ini—Telegram, web, suara, dan Android—tidak bersaing satu sama lain, melainkan berfungsi sebagai jalur yang berbeda untuk mencapai satu tujuan: interaksi yang mulus antara pengguna dan drone. Agen Hermes bertindak sebagai pusat pengelolaan semua masukan dari antarmuka ini, menerjemahkannya menjadi perintah yang koheren, dan kemudian menugaskannya kepada PicoClaw untuk dieksekusi. Misalnya, perintah "Take off" yang dikirim melalui Telegram akan diproses oleh Hermes, yang kemudian memerintahkan PicoClaw untuk mengaktifkan motor hingga mencapai ketinggian yang aman. Sementara itu, agen PicoClaw secara simultan akan mengelola kontrol keseimbangan dan stabilitas drone. Dengan arsitektur ini, Nangroe OS AI

memastikan bahwa pengguna dapat mengontrol drone dengan cara yang paling nyaman bagi mereka, tanpa harus memahami kompleksitas di balik layar.

## Analisis Ketahanan, Risiko Teknis, dan Rekomendasi Kerangka Penelitian

Meskipun visi Nanggro OS AI sangat ambisius dan memiliki potensi dampak yang signifikan, implementasinya dihadapkan pada serangkaian tantangan teknis dan risiko yang perlu dianalisis secara mendalam. Analisis ini akan mengidentifikasi risiko utama, menyoroti kesenjangan penelitian, dan memberikan rekomendasi kerangka penelitian yang terstruktur untuk memitigasi tantangan tersebut.

Salah satu risiko teknis terbesar adalah **kinerja AI on-device**. Menjalankan model LLM seperti Phi-3-mini atau model deteksi objek kompleks pada perangkat dengan sumber daya terbatas (RAM, CPU) akan sangat lambat dan mungkin tidak dapat memenuhi persyaratan waktu nyata untuk tugas-tugas kritis seperti penghindaran rintangan atau kontrol keseimbangan. Latensi inferensi yang tinggi dapat menyebabkan respons drone yang tertunda, yang berbahaya dalam situasi dinamis. Selain itu, penggunaan memori yang besar oleh model AI dapat mengurangi sumber daya yang tersedia untuk sistem operasi dan tugas-tugas lainnya, meningkatkan risiko kegagalan sistem. Mitigasi untuk risiko ini melibatkan optimasi yang ekstrem, termasuk penggunaan model yang sangat terkuantisasi (misalnya, 4-bit atau 8-bit integer), pengembangan runtime inferensi yang sangat dioptimalkan, dan mungkin penggunaan hardware accelerator khusus jika memungkinkan.

Risiko teknis kedua adalah **keandalan sensor dan interferensi elektromagnetik (EMI)**. Operasi drone, terutama di lingkungan dengan banyak sumber gangguan seperti motor listrik, dapat menghasilkan EMI yang kuat. EMI ini dapat mengganggu sinyal GPS, menyebabkan drift posisi yang signifikan, dan mengganggu komunikasi nirkabel seperti sinyal radio kontrol dan modul GSM <sup>8</sup>. Gangguan pada GPS dapat membuat fitur navigasi dan RTH gagal. Selain itu, sensor lain seperti magnetometer (untuk penentuan arah) sangat rentan terhadap EMI dari medan magnet yang dihasilkan oleh arus listrik besar ke motor. Mitigasi yang diperlukan termasuk desain PCB yang cermat dengan jalur sinyal yang terisolasi, penggunaan filter pasif dan aktif, serta mungkin pemasangan shield logam di sekitar komponen yang paling sensitif. Kalibrasi sensor secara berkala dan penggunaan algoritma penggabungan sensor (sensor fusion) yang kuat, seperti

Extended Kalman Filter (EKF), akan menjadi krusial untuk memfilter noise dan gangguan dari data mentah sensor.

Kesenjangan arsitektur yang paling signifikan adalah **kesenjangan antara kebutuhan komputasi AI yang tinggi dan kapabilitas perangkat keras Arduino yang relatif sederhana**. Arduino Uno, misalnya, hanya memiliki 2KB RAM dan kecepatan clock 16MHz, yang jelas tidak cukup untuk menjalankan LLM atau model AI kompleks. Oleh karena itu, kerangka penelitian ini harus secara eksplisit beralih ke penggunaan mikrokontroler yang lebih kuat sebagai unit pemrosesan utama. Chip SoC seperti ESP32-S3 adalah pilihan yang jauh lebih logis, karena ia memiliki CPU dual-core, RAM yang lebih besar (misalnya, 520KB SRAM + PSRAM), dan dukungan bawaan untuk USB OTG, Wi-Fi, Bluetooth, serta konektivitas USB [19](#) [27](#). Dalam arsitektur yang direvisi, ESP32-S3 akan bertindak sebagai "cerdas" yang menjalankan Nanggro OS AI, berkomunikasi dengan Arduino (sebagai "tangan" yang lebih sederhana) melalui I2C atau UART untuk kontrol motorik yang sangat cepat. Ini adalah titik pergeseran arsitektur yang fundamental untuk membuat proyek ini layak secara teknis.

Berdasarkan analisis ini, berikut adalah rekomendasi kerangka penelitian terstruktur dalam beberapa tahap:

### Tahap 1: Pembangunan Prototipe Dasar dan Validasi Komponen

- **Tujuan:** Membangun dan menguji integrasi perangkat keras dasar serta implementasi kontrol manual.
- **Aktivitas:**
  1. Merakit drone tiga baling dasar dengan menggunakan ESP32-S3 sebagai ko
  2. Mengintegrasikan sensor GPS (idealnya RTK) dan kamera.
  3. Menguji kontrol keseimbangan dan penerbangan manual menggunakan remote
  4. Membangun dan menguji lapisan deteksi hardware otomatis sederhana yang

**\*\*Tahap**base yang dapat dengan mudah dibackup atau disinkronkan.

Sinkronisasi ke cloud bukanlah proses yang terus-menerus, melainkan sebuah proses yang diaktifkan secara eksplisit atau terjadi secara periodik saat koneksi tersedia. Arsitektur sinkronisasi harus dirancang untuk mengatasi tantangan seperti koneksi yang tidak stabil dan bandwidth yang terbatas. Pendekatan yang paling efisien adalah *delta synchronization*, di mana hanya perubahan (delta) yang dikirimkan ke server, bukan seluruh database. Sebagai contoh, jika sebuah misi telah dijalankan dan menghasilkan 1000 foto, sistem tidak akan mengunggah semua foto tersebut. Sebaliknya, ia akan mengunggah metadata misi (koordinat, waktu, parameter), serta daftar hash dari file

foto. Server kemudian dapat memeriksa hash tersebut dan hanya meminta file yang belum dimilikinya. Ini secara dramatis mengurangi volume data yang harus dikirimkan dan mempercepat proses sinkronisasi.

Studi tentang sistem penyimpanan yang terdistribusi menunjukkan bahwa arsitektur yang memisahkan lapisan penyimpanan dari lapisan aplikasi dapat meningkatkan kinerja secara signifikan <sup>42</sup>. Dalam konteks ini, Nangroe OS AI dapat mengadopsi pendekatan serupa dengan memisahkan modul penyimpanan data dari modul perencanaan dan eksekusi. Modul penyimpanan akan bertanggung jawab untuk semua operasi I/O (input/output) ke disk dan manajemen cache, sementara modul lainnya hanya berkomunikasi dengan modul penyimpanan melalui API yang terdefinisi dengan baik. Ini memungkinkan pengoptimalan modul penyimpanan secara independen, misalnya dengan menerapkan algoritma kompresi data atau enkripsi end-to-end untuk keamanan data.

## **Dukungan Daya Darurat dan Manajemen Energi: Panel Surya sebagai Jaminan Kelangsungan Hidup**

Dukungan daya darurat dari panel surya kecil adalah fitur ketahanan yang sangat praktis dan penting <sup>2</sup>. Dalam konteks operasi di lapangan, baterai utama drone dapat habis sebelum misi selesai, atau sistem komando dapat kehilangan daya. Panel surya kecil, dikombinasikan dengan sistem manajemen baterai (BMS) yang cerdas, dapat berfungsi sebagai "jaminan kelangsungan hidup" (*life insurance*) untuk sistem.

Sistem manajemen baterai harus dirancang untuk mengoptimalkan penggunaan energi dari semua sumber yang tersedia: baterai utama, baterai cadangan, dan panel surya. Sistem harus memiliki logika cerdas untuk menentukan kapan harus mengisi baterai, kapan harus mengalihkan beban ke sumber daya alternatif, dan kapan harus memasuki mode hemat daya (*low-power mode*). Sebagai contoh, ketika baterai utama turun di bawah 20%, sistem dapat secara otomatis mengaktifkan panel surya untuk mengisi baterai cadangan yang menggerakkan modul komunikasi GSM dan GPS. Ini memastikan bahwa sistem tetap dapat dikendalikan dan dilacak, bahkan jika baterai utama sudah tidak dapat menggerakkan motor.

Desain sistem pengisian baterai surya juga harus memperhitungkan efisiensi. Implementasi Maximum Power Point Tracking (MPPT) adalah teknik yang sangat penting untuk memastikan bahwa panel surya beroperasi pada titik daya maksimumnya, terlepas dari kondisi pencahayaan yang berubah-ubah <sup>2</sup>. Sistem MPPT dapat diimplementasikan menggunakan mikrokontroler khusus atau bahkan diintegrasikan ke dalam firmware Nangroe OS AI jika mikrokontroler utama memiliki ADC (Analog-to-



Digital Converter) yang cukup presisi untuk memantau tegangan dan arus panel. Studi tentang sistem pengisian baterai surya untuk rumah tangga di pedesaan menunjukkan bahwa desain BMS yang baik dapat secara signifikan memperpanjang umur baterai dan meningkatkan keandalan sistem secara keseluruhan <sup>2</sup>.

## Fitur Keselamatan Kritis: Return-to-Home dan Autopilot sebagai Jaminan Keandalan

Fitur keselamatan kritis seperti *Return-to-Home* (RTH) dan *autopilot* saat kehilangan kendali remote adalah elemen yang tidak dapat dikompromikan dalam desain sistem. Kegagalan dalam fitur-fitur ini tidak hanya dapat menyebabkan kehilangan perangkat yang mahal, tetapi juga dapat menimbulkan risiko keselamatan bagi orang dan properti di sekitarnya. Oleh karena itu, implementasi fitur-fitur ini harus berada di lapisan PicoClaw, yang merupakan lapisan kontrol deterministik dan real-time <sup>1</sup>.

Logika RTH harus didasarkan pada beberapa trigger yang independen dan redundan: 1. **Kehilangan Sinyal Remote:** Sistem harus memantau kekuatan sinyal dari remote control (misalnya, modul HC-12) secara terus-menerus. Jika sinyal turun di bawah ambang batas tertentu selama periode waktu yang ditentukan, RTH akan diaktifkan. 2. **Baterai Rendah:** Sistem harus memiliki dua ambang batas baterai: ambang batas "peringatan" (misalnya, 25%) yang memberi tahu pengguna, dan ambang batas "kritis" (misalnya, 15%) yang memicu RTH secara otomatis. 3. **Kegagalan Sensor:** Jika sistem mendeteksi kegagalan kritis pada sensor navigasi, seperti hilangnya sinyal GPS atau kegagalan IMU, RTH harus diaktifkan sebagai tindakan pencegahan.

Ketika RTH diaktifkan, PicoClaw harus segera mengambil alih kendali. Prosesnya harus sangat deterministik: (1) drone akan menaikkan ketinggian ke ketinggian aman (misalnya, 50 meter) untuk menghindari rintangan, (2) drone akan menghitung jalur paling langsung ke titik home (koordinat GPS saat take-off), dan (3) drone akan terbang ke titik tersebut dan mendarat secara otomatis. Semua langkah ini harus dihitung dan dieksekusi oleh PicoClaw tanpa ketergantungan pada Hermes atau LLM, memastikan respons yang cepat dan andal.

Demikian pula, *autopilot* saat kehilangan kendali remote harus merupakan fungsi lapisan bawah. Jika sinyal dari remote benar-benar terputus, PicoClaw harus segera beralih ke mode *hover* stabil, mempertahankan ketinggian dan posisi saat ini dengan menggunakan data dari IMU dan altimeter. Ini memberikan waktu bagi pengguna untuk memperbaiki koneksi atau memicu RTH secara manual. Kestabilan hover ini adalah bukti keandalan lapisan kontrol taktis dan merupakan jaminan keamanan dasar yang tidak boleh dikompromikan.

# Analisis Komprehensif: Lubang Penelitian, Risiko Teknis, dan Strategi Implementasi

Meskipun kerangka penelitian ini menawarkan visi yang sangat menarik dan komprehensif, analisis mendalam terhadap materi yang tersedia mengungkapkan beberapa lubang penelitian kritis, risiko teknis yang signifikan, dan tantangan implementasi yang harus diatasi secara strategis. Mengakui dan merencanakan mitigasi terhadap tantangan-tantangan ini bukanlah tanda kelemahan, melainkan bukti kedalaman dan kematangan pendekatan penelitian.

## Identifikasi Lubang Penelitian Utama: Jembatan antara Visi dan Realitas Hardware

Salah satu lubang penelitian terbesar dan paling fundamental adalah kesenjangan antara visi arsitektur perangkat lunak yang sangat canggih dan kenyataan keterbatasan perangkat keras yang digunakan. Visi Nanggro OS AI—dengan deteksi hardware otomatis, multi-agent system, dan LLM on-device—secara inheren menuntut sumber daya komputasi yang jauh melampaui kapabilitas mikrokontroler Arduino klasik seperti Uno atau Nano <sup>1</sup>. Arduino Uno, misalnya, hanya memiliki 2 KB RAM dan 32 KB flash memory, yang jauh dari cukup untuk menjalankan bahkan model LLM terkecil sekalipun.

Oleh karena itu, kerangka penelitian ini harus secara eksplisit mengadopsi pendekatan *heterogeneous computing*, yaitu penggunaan beberapa jenis prosesor yang berbeda, masing-masing dioptimalkan untuk tugas yang berbeda. Rekomendasi praktis yang paling kuat adalah menggunakan ESP32-S3 sebagai *main processor* atau *gateway*. ESP32-S3 memiliki 512 KB RAM dan 8 MB flash memory, serta dukungan USB OTG bawaan, yang membuatnya menjadi kandidat ideal untuk menjalankan lapisan Hermes, LLM lokal, dan antarmuka komunikasi <sup>19 44</sup>. Sementara itu, mikrokontroler Arduino yang lebih sederhana dapat tetap digunakan sebagai *flight controller* khusus, yang bertugas menjalankan algoritma PID untuk kontrol motor dan algoritma penghindaran rintangan sederhana, yang semuanya berjalan pada lapisan PicoClaw <sup>6</sup>. Komunikasi antara ESP32-S3 dan Arduino dapat dilakukan melalui antarmuka serial atau I2C, dengan ESP32-S3 bertindak sebagai master dan Arduino sebagai slave.

Lubang penelitian lainnya adalah dalam hal *hardware abstraction layer* (HAL) untuk deteksi otomatis. Meskipun konsepnya jelas, implementasi spesifik untuk berbagai protokol (USB, I2C, SPI) pada platform ESP32-S3 memerlukan pengembangan driver yang mendalam. Literatur menunjukkan bahwa ESP-IDF menyediakan stack USB host dan driver kelas USB, tetapi integrasi penuh dengan sistem operasi robotika masih

merupakan area penelitian aktif [32](#). Oleh karena itu, kerangka penelitian harus mencakup pengembangan HAL yang modular, di mana setiap driver untuk jenis perangkat (kamera, sensor, aktuator) dapat dikembangkan dan diuji secara independen.

## **Analisis Risiko Teknis: Menghadapi Batasan Nyata di Dunia Nyata**

Risiko teknis utama yang dihadapi dalam implementasi kerangka ini dapat dikategorikan menjadi tiga kelompok: risiko kinerja, risiko keandalan, dan risiko integrasi.

**Risiko Kinerja:** Risiko paling dominan adalah kinerja inferensi LLM on-device. Meskipun model Phi-3-mini dalam format GGUF sangat menjanjikan, inferensi pada perangkat dengan RAM terbatas akan menghasilkan latensi yang signifikan. Jika LLM membutuhkan 5-10 detik untuk memproses sebuah perintah, maka pengalaman pengguna akan menjadi sangat buruk, dan sistem tidak akan dapat digunakan untuk aplikasi yang membutuhkan respons cepat. Tinjauan komprehensif tentang penyebaran LLM pada perangkat sumber daya terbatas secara eksplisit menyebutkan bahwa latensi tinggi dan konsumsi daya besar adalah tantangan utama [41](#). Mitigasi risiko ini harus berfokus pada optimasi ekstrem: quantization ke 4-bit, penggunaan cache KV yang sangat efisien, dan penggunaan model yang lebih kecil untuk tugas-tugas rutin.

**Risiko Keandalan:** Operasi multi-moda (udara, darat, air) menimbulkan risiko keandalan yang unik. Setiap mode operasi menempatkan stres yang berbeda pada sistem. Transisi dari udara ke darat, misalnya, memerlukan penyesuaian instan dalam kontrol stabilitas dan pengelolaan daya. Jika sistem gagal mendeteksi permukaan tanah dengan akurat, drone dapat mendarat dengan kecepatan yang terlalu tinggi dan rusak. Demikian pula, operasi di air memerlukan desain tahan air yang sempurna untuk semua komponen elektronik, yang merupakan tantangan insinyur yang signifikan.

**Risiko Integrasi:** Risiko terbesar dalam integrasi adalah *Electromagnetic Interference* (EMI). Seperti yang diidentifikasi dalam studi kasus sistem sensor drone, EMI dari motor drone dapat mengganggu komunikasi nirkabel dan kinerja sensor secara signifikan [8](#). Ini bukan masalah teoretis; ini adalah tantangan fisik yang nyata yang harus diatasi melalui desain PCB yang cermat, pemilihan komponen, dan pengujian yang ekstensif. Kegagalan dalam mengatasi EMI dapat menyebabkan kehilangan koneksi GSM, kegagalan GPS, atau bahkan kehilangan kendali total.

## Strategi Implementasi Bertahap: Dari Prototipe Dasar ke Sistem Lengkap

Menghadapi kompleksitas dan risiko yang tinggi, strategi implementasi yang paling bijaksana adalah pendekatan bertahap (*phased development*). Ini memungkinkan tim penelitian untuk memvalidasi asumsi, mengidentifikasi masalah lebih awal, dan membangun kepercayaan pada sistem secara bertahap.

**Tahap 1: Prototipe Dasar dan Deteksi Hardware.** Fokus pada integrasi perangkat keras dasar: drone tiga baling, ESP32-S3, modul GPS, dan kamera sederhana. Tujuan utama adalah membangun lapisan deteksi hardware otomatis untuk komponen-komponen ini dan memverifikasi bahwa sistem dapat mengenali dan menginisialisasi mereka secara otomatis. Hasil akhirnya adalah sebuah sistem yang dapat mengambil alih kendali dari remote dan terbang secara stabil dalam mode manual.

**Tahap 2: Autonomi Dasar dan Komunikasi.** Pada tahap ini, fokus beralih ke implementasi fungsi autonomi dasar: Return-to-Home, auto-balancing, dan obstacle avoidance sederhana menggunakan sensor ultrasonik. Secara paralel, sistem komunikasi melalui Telegram bot dan dashboard web harus dikembangkan dan diuji. Hasil akhirnya adalah sistem yang dapat dikendalikan sepenuhnya dari jarak jauh melalui Telegram dan dapat kembali ke titik awal secara otomatis.

**Tahap 3: Integrasi Nangroe OS AI dan LLM.** Tahap ini adalah inti dari penelitian. Di sini, arsitektur multi-agen Hermes-PicoClaw dibangun, dan model LLM Phi-3-mini dalam format GGUF diintegrasikan. Fokus utama adalah pada kemampuan sistem untuk memahami perintah natural language dan mengubahnya menjadi rencana penerbangan. Hasil akhirnya adalah sistem yang dapat menerima perintah seperti "terbang ke koordinat X dan kembali" dan mengeksekusinya secara otonom.

**Tahap 4: Aplikasi Pertanian Presisi.** Pada tahap ini, sistem diuji dalam skenario pertanian nyata. Ini mencakup pengujian pemetaan sawah untuk akurasi GSD dan pengukuran luasan, serta pengujian mekanisme pelepas pupuk untuk presisi penempatan dan dosis. Hasil akhirnya adalah sistem yang dapat menghasilkan peta lahan yang akurat dan menabur pupuk sesuai dengan rencana yang telah dikalkulasi.

**Tahap 5: Optimasi dan Adaptasi Lanjutan.** Tahap akhir ini berfokus pada peningkatan kinerja, efisiensi daya, dan pengembangan fitur self-learning yang lebih canggih, seperti adaptasi model penghindaran rintangan berdasarkan pengalaman operasional.

Dengan mengikuti strategi bertahap ini, kerangka penelitian tidak hanya menjadi sebuah dokumen konseptual, tetapi juga menjadi sebuah peta jalan yang dapat ditindaklanjuti, yang memandu pengembangan sistem dari ide abstrak menjadi solusi nyata yang dapat mengubah wajah pertanian di Indonesia.

## **Kesimpulan: Menuju Sistem Otonom yang Benar-Benar Terintegrasi dan Berdampak**

Kerangka penelitian yang telah dianalisis secara mendalam dalam laporan ini bukanlah sekadar rencana teknis untuk membangun sebuah drone. Ia adalah sebuah visi transformasional untuk menciptakan sebuah *platform robotika otonom* yang berakar pada prinsip-prinsip ketahanan, keterjangkauan, dan kemanfaatan langsung bagi masyarakat. Inti dari visi ini adalah Nanggroe OS AI, sebuah sistem operasi yang dirancang bukan untuk menjadi "otak" tunggal, tetapi sebagai sebuah *sistem saraf pusat* yang cerdas dan adaptif, yang menghubungkan perangkat keras fisik dengan kecerdasan buatan on-device dan kebutuhan manusia di lapangan.

Analisis komprehensif telah menunjukkan bahwa keberhasilan kerangka ini tidak ditentukan oleh kecanggihan satu komponen, melainkan oleh keberhasilan integrasi holistik antara semua lapisannya. Deteksi perangkat keras otomatis bukanlah fitur yang berdiri sendiri; ia adalah fondasi yang memungkinkan arsitektur multi-agen Hermes-PicoClaw beroperasi secara dinamis. Kemampuan LLM untuk memahami bahasa alami bukanlah tujuan akhir; ia adalah alat yang memungkinkan sistem untuk berkomunikasi secara alami dan membuat keputusan strategis yang kompleks. Dan aplikasi pertanian presisi bukanlah sekadar demo teknis; ia adalah bukti nyata bahwa teknologi ini dapat mengatasi tantangan nyata—seperti efisiensi penggunaan pupuk dan akurasi pemetaan lahan—yang secara langsung memengaruhi kesejahteraan petani dan keberlanjutan lingkungan.

Yang paling penting, kerangka ini secara eksplisit mengakui dan merancang untuk realitas operasional di dunia nyata. Prinsip *offline-first*, dukungan daya darurat dari panel surya, dan fitur keselamatan kritis seperti Return-to-Home bukanlah fitur tambahan; mereka adalah pilar utama yang menjamin bahwa sistem ini tidak hanya berfungsi di laboratorium, tetapi juga dapat diandalkan di lahan pertanian pedesaan yang jauh dari infrastruktur teknologi modern. Ini adalah perbedaan mendasar antara sebuah proyek akademik dan sebuah solusi yang benar-benar berdampak.

Oleh karena itu, kesimpulan akhir dari laporan ini adalah sebuah ajakan untuk bertindak. Kerangka penelitian ini telah memberikan fondasi konseptual dan analitis yang kokoh. Langkah selanjutnya adalah memulai implementasi bertahap, dengan fokus pada validasi asumsi teknis dan pengujian iteratif di lingkungan yang paling dekat dengan kondisi operasional target. Dengan mengikuti peta jalan yang telah diuraikan—dari pembangunan prototipe dasar hingga pengembangan fitur adaptasi lanjutan—penelitian ini memiliki potensi yang sangat besar untuk melampaui ekspektasi dan menjadi tonggak sejarah dalam pengembangan sistem otonom yang benar-benar terintegrasi dan berorientasi pada manusia.

---

en

1. [https://cdn.qwenlm.ai/3a20d8d7-1ab4-4df8-abdd-8af096723b9c/51d62d94-d33f-4829-98be-369b1ba668f0\\_Nanggroee\\_OS\\_AI\\_Blueprint\\_Package.pdf?key=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZV91c2VyaXZlIjoiM2EyMGQ4ZDctMWFhNC00ZGY4LWFiZGQtOGFmMDk2NzIzYjIiwicmVzb3VyY2VfaWQiOiI1MWQ2MmQ5NC1kMzNmLTQ4MjktOThiZS0zNjliMWJhNjY4ZjAiLCJyZXNvdXJjZV9jaGFOX2lkIjpudWxsQzJwU39KzSY5KJgq3GaSxTqCAum6Q-xcVfmnpf\\_E1rk](https://cdn.qwenlm.ai/3a20d8d7-1ab4-4df8-abdd-8af096723b9c/51d62d94-d33f-4829-98be-369b1ba668f0_Nanggroee_OS_AI_Blueprint_Package.pdf?key=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZV91c2VyaXZlIjoiM2EyMGQ4ZDctMWFhNC00ZGY4LWFiZGQtOGFmMDk2NzIzYjIiwicmVzb3VyY2VfaWQiOiI1MWQ2MmQ5NC1kMzNmLTQ4MjktOThiZS0zNjliMWJhNjY4ZjAiLCJyZXNvdXJjZV9jaGFOX2lkIjpudWxsQzJwU39KzSY5KJgq3GaSxTqCAum6Q-xcVfmnpf_E1rk)
2. Arduino based solar powered battery charging system for rural SHS <https://www.semanticscholar.org/paper/Arduino-based-solar-powered-battery-charging-system-Kaur-Gambhir/c2dca65332d451f3280167e1d7858cd9262ce016>
3. Design of UAV Autonomous Charging Pad for Surveillance [https://www.researchgate.net/publication/368789668\\_Design\\_of\\_UAV\\_Autonomous\\_Charging\\_Pad\\_for\\_Surveillance](https://www.researchgate.net/publication/368789668_Design_of_UAV_Autonomous_Charging_Pad_for_Surveillance)
4. Speech Recognition With an Arduino Nano - Instructables <https://www.instructables.com/Speech-Recognition-With-an-Arduino-Nano/>
5. Personal Assistant With Telegram & Arduino. : 9 Steps - Instructables <https://www.instructables.com/Personal-Assistant-With-Telegram-Arduino/>
6. MicroROS on RT-Thread - 知乎专栏 <https://zhuanlan.zhihu.com/p/416498093>
7. Navigation and Deployment of Solar-Powered Unmanned Aerial ... <https://www.mdpi.com/2504-446X/8/2/42>
8. Autonomous Agricultural Monitoring with Aerial Drones and RF ... <https://arxiv.org/html/2502.16028v1>
9. Mapping paddy rice planting area in northeastern Asia with Landsat ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC5181848/>

10. In-Field Rice Spike Detection Based on Improved YOLOv5 and ... <https://www.mdpi.com/2073-4395/14/4/836>
11. Using Drones to Predict Degradation of Surface Drainage on ... - MDPI <https://www.mdpi.com/2624-7402/7/4/112>
12. How Accurate Are Drone Surveys? What You Need to Know <https://www.zenatch.com/drone-survey-accuracy-guide/>
13. Long Range Dropping System for Drones With Arduino ... <https://www.instructables.com/Long-Range-Dropping-System-for-Drones-With-Arduino/>
14. LLM as a System Service on Mobile Devices - arXiv <https://arxiv.org/html/2403.11805v1>
15. Key Feature - Docs | openEuler [https://docs.openeuler.org/en/docs/25.09/server/quickstart/releasenotes/key\\_features.html](https://docs.openeuler.org/en/docs/25.09/server/quickstart/releasenotes/key_features.html)
16. Phi-3-mini-4k-instruct-gguf从零开始：下载GGUF、配置vLLM [https://blog.csdn.net/weixin\\_28713083/article/details/156589360](https://blog.csdn.net/weixin_28713083/article/details/156589360)
17. Generative AI with Phi-3-mini: A Guide to Inference and Deployment <https://techcommunity.microsoft.com/blog/azuredevcommunityblog/getting-started---generative-ai-with-phi-3-mini-a-guide-to-inference-and-deploym/4121315>
18. ESP32-S3 + TensorFlow Lite Micro: A Practical Guide to Local Wake ... <https://dev.to/zediot/esp32-s3-tensorflow-lite-micro-a-practical-guide-to-local-wake-word-edge-ai-inference-5540>
19. 13. ESP32-S3 特色功能探索：AI/ML 边缘与USB-OTG - 知乎专栏 <https://zhuanlan.zhihu.com/p/1961432312261632691>
20. ROMR: A ROS-based open-source mobile robot - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC10197097/>
21. ROS (Robot Operating System) quickstart guide <https://docs.ultralytics.com/guides/ros-quickstart>
22. [PDF] Plug-and-play Physical Computing with Jacdac - Microsoft [https://www.microsoft.com/en-us/research/wp-content/uploads/2022/11/Jacdac\\_IMWUT\\_V6\\_Issue3.pdf](https://www.microsoft.com/en-us/research/wp-content/uploads/2022/11/Jacdac_IMWUT_V6_Issue3.pdf)
23. AWM: An Open-Source ML-based Robotics Perception Framework ... <https://arxiv.org/html/2506.00645v1>
24. Running LLMs On-Device in Android: GGUF Models, NNAPI, and ... [https://dev.to/software\\_mvfp-factory/running-llms-on-device-in-android-gguf-models-nnapi-and-the-real-performance-tradeoffs-5bfc](https://dev.to/software_mvfp-factory/running-llms-on-device-in-android-gguf-models-nnapi-and-the-real-performance-tradeoffs-5bfc)
25. Espressif Education <https://www.espressif.com/en/ecosystem/education>
26. Modular-Things: Plug-and-Play with Virtualized Hardware <https://dl.acm.org/doi/10.1145/3544549.3585642>



27. USB 方案 | 乐鑫科技 <https://www.espressif.com/zh-hans/solutions/peripherals/usb-solutions>
28. 商品搜索 - 微雪电子 [https://www.waveshare.net/search.html?kw=UsB&pno=2&c\\_code=10011008](https://www.waveshare.net/search.html?kw=UsB&pno=2&c_code=10011008)
29. Artificial Intelligence - arXiv <https://arxiv.org/list/cs.AI/new>
30. Simple Index - SUSTech Open Source Mirrors <https://mirrors.sustech.edu.cn/pypi/simple/>
31. RoboOS: A Hierarchical Embodied Framework for Cross ... - arXiv <https://arxiv.org/html/2505.03673v1>
32. USB Host Solution - - — ESP-IoT-Solution release-v2.0 documentation [https://docs.espressif.com/projects/esp-iot-solution/en/release-v2.0/usb/usb\\_overview/usb\\_host\\_solutions.html](https://docs.espressif.com/projects/esp-iot-solution/en/release-v2.0/usb/usb_overview/usb_host_solutions.html)
33. RoboNeuron: A Modular Framework Linking Foundation Models ... <https://arxiv.org/html/2512.10394v1>
34. LLM Robotics Demo - Intel® Edge Controls for Industrial [https://eci.intel.com/embodied-sdk-docs/content/sample\\_pipelines/llm\\_robotics.html](https://eci.intel.com/embodied-sdk-docs/content/sample_pipelines/llm_robotics.html)
35. Bringing robotics middleware onto tiny microcontrollers | Bosch Global <https://www.bosch.com/stories/bringing-robotics-middleware-onto-tiny-microcontrollers/>
36. Towards LLM-Driven State Machines for Runtime Robot Behavior in ... [https://www.researchgate.net/publication/404313895\\_Towards\\_LLM-Driven\\_State\\_Machines\\_for\\_Runtime\\_Robot\\_Behavior\\_in\\_ROS\\_2](https://www.researchgate.net/publication/404313895_Towards_LLM-Driven_State_Machines_for_Runtime_Robot_Behavior_in_ROS_2)
37. ESP32-S3-USB-OTG en | 乐鑫科技 <https://www.espressif.com/zh-hans/dev-board/esp32-s3-usb-otg-en>
38. ESP USB 外设介绍- - — ESP-IoT-Solution latest 文档 [https://docs.espressif.com/projects/esp-iot-solution/zh\\_CN/latest/usb/usb\\_overview/usb\\_overview.html](https://docs.espressif.com/projects/esp-iot-solution/zh_CN/latest/usb/usb_overview/usb_overview.html)
39. 通过USB 升级设备固件- ESP32-S3 - — ESP-IDF 编程指南v5.0.4 文档 [https://docs.espressif.com/projects/esp-idf/zh\\_CN/v5.0.4/esp32s3/api-guides/dfu.html](https://docs.espressif.com/projects/esp-idf/zh_CN/v5.0.4/esp32s3/api-guides/dfu.html)
40. On-device Online Learning and Semantic Management of TinyML ... <https://arxiv.org/html/2405.07601v2>
41. On-Device Language Models: A Comprehensive Review - arXiv <https://arxiv.org/html/2409.00088v1>
42. 操作系统/编程语言/数据库2026\_4\_16 - arXiv每日学术速递 <http://www.arxivdaily.com/thread/78861>
43. USB-OTG 外设介绍- - — ESP-IoT-Solution latest 文档 [https://docs.espressif.com/projects/esp-iot-solution/zh\\_CN/latest/usb/usb\\_overview/usb\\_otg.html](https://docs.espressif.com/projects/esp-iot-solution/zh_CN/latest/usb/usb_overview/usb_otg.html)

44. ESP32-S3-USB-OTG - ESP32-S3 - — esp-dev-kits latest 文档 [https://docs.espressif.com/projects/esp-dev-kits/zh\\_CN/latest/esp32s3/esp32-s3-usb-otg/user\\_guide.html](https://docs.espressif.com/projects/esp-dev-kits/zh_CN/latest/esp32s3/esp32-s3-usb-otg/user_guide.html)
45. 27- ESP32-S3 USB虚拟串口 (USB-OTG 外设介绍 ... - CSDN博客 [https://blog.csdn.net/m0\\_60134435/article/details/138873405](https://blog.csdn.net/m0_60134435/article/details/138873405)
46. RoboNeuron: A Middle-Layer Infrastructure for Agent-Driven ... - arXiv <https://arxiv.org/abs/2512.10394>
47. Robotics - arXiv <https://arxiv.org/list/cs.RO/new>
48. ESP32 是否支持USB 功能? [https://documentation.espressif.com/projects/esp-faq/zh\\_CN/latest/software-framework/peripherals/usb.html?](https://documentation.espressif.com/projects/esp-faq/zh_CN/latest/software-framework/peripherals/usb.html?)
49. 告别串口转换芯片! 用ESP32-S3的USB-OTG直接烧录固件全指南 [https://blog.csdn.net/weixin\\_29194817/article/details/158746406](https://blog.csdn.net/weixin_29194817/article/details/158746406)